



VS Code Agent AI Execution Spec: CUDA-Optimized RL Pipeline

Target Hardware: NVIDIA GeForce RTX 3070 Ti (8GB VRAM) | OS: Linux / Ubuntu 22.04

Architecture: PPO + Vectorized GPU Env

Max VRAM Target: < 6.5 GB

Target Throughput: > 15,000 FPS

一、Agent 指令指南 (Prompt Directive for AI Agent)

這份檔案是針對 VS Code Coding Agent (如 Cursor, Cline, GitHub Copilot Workspace) 的專案建置規格書。請 Agent 嚴格遵守以下規則與順序逐步執行專案初始化與程式碼生成：

- 專案架構**：建立輕量化、模組化的 PyTorch + CleanRL 架構，嚴格禁止冗餘依賴。
- 記憶體管理**：針對 8GB VRAM 限制，所有 Tensor 必須嚴格使用 `torch.float32` 或開啟 AMP `torch.cuda.amp.autocast()`，避免 OOM。
- 執行順序**：按照【階段 1】至【階段 4】依序建立檔案、執行測試並進行驗證。

二、專案架構與目錄結構 (Project Structure)

請 Agent 建立以下檔案目錄結構：

```
rl_3070ti_project/  
├─ requirements.txt  
├─ config.yaml  
├─ src/  
│   ├── __init__.py  
│   ├── envs.py           # 平行化向量環境封裝  
│   ├── model.py         # Actor-Critic 網路 (PyTorch)  
│   └─ agent.py          # PPO 演算法核心與 CUDA 優化  
├─ train.py              # 主訓練入口腳本  
└─ benchmark.py          # GPU 效能與 VRAM 檢測腳本
```

三、依賴套件配置 (requirements.txt)

```
torch>=2.0.0  
torchvision  
gymnasium[classic_control,box2d]>=0.29.0  
numpy<2.0.0  
pyyaml  
tensorboard  
tqdm  
weasyprint
```

四、核心優化設定 (config.yaml)

此設定檔已為 RTX 3070 Ti (8GB VRAM / 6144 CUDA Cores) 完成參數最佳化：

```
hardware:
  device: "cuda"
  enable_amp: true          # 啟用自動混合精度 (AMP) 壓榨 Tensor Core
  compile_model: true       # 啟用 torch.compile() 提高運算效率

environment:
  env_id: "BipedalWalker-v3"
  num_envs: 32              # 32 個平行採樣環境 (兼顧 CPU/GPU 搬移效率)
  gamma: 0.99

hyperparameters:
  learning_rate: 3.0e-4
  num_steps: 2048           # 每個環境採樣步數 (Rollout Buffer Size = 32 * 2048 = 65,536)
  batch_size: 2048          # 大 Batch 填滿 CUDA 核心
  n_epochs: 10
  clip_coef: 0.2
  ent_coef: 0.001
  max_grad_norm: 0.5
```

五、逐步落地指令 (Step-by-Step Execution Plan)

【階段 1】環境初始化與硬體檢測

- **任務 A**：建立虛擬環境並安裝 `requirements.txt`。
- **任務 B**：建立 `benchmark.py`，檢查 PyTorch 是否成功識別 RTX 3070 Ti，並測試可用的 VRAM 與 Tensor Core 算力。

【階段 2】核心模型與向量環境開發 (src/)

- **任務 A (src/envs.py)**：使用 `gymnasium.vector.AsyncVectorEnv` 封裝 32 個平行環境，實作連續動作空間的預處理。
- **任務 B (src/model.py)**：撰寫 Actor-Critic 神經網路，正則化初始化權重 (Orthogonal Initialization)，並確保架構可被 `torch.compile()` 編譯。
- **任務 C (src/agent.py)**：實作 PPO 演算法更新邏輯，加入 `torch.cuda.amp.autocast()` 混合精度計算與 GAE (Generalized Advantage Estimation)。

【階段 3】主訓練腳本與監控 (train.py)

- 任務 A：整合配置檔 `config.yaml`，編寫主訓練迴圈。
- 任務 B：加入 TensorBoard 數據記錄（記錄 FPS、Loss、Reward、VRAM Usage）。
- 任務 C：加入自動存檔機制 (Checkpointing)，每當獲得最高 Reward 時儲存模型至 `checkpoints/best_model.pt`。

【階段 4】驗證與 Performance Profiling

- 任務 A：執行 `python train.py`，確保訓練吞吐量達到 ****10,000+ FPS****。
- 任務 B：觀察 VRAM 使用量，確認控制在 **4GB ~ 6GB** 之間，確保極致利用且不發生 OOM。

六、效能指標檢核表 (Acceptance Criteria)

檢核項目	目標指標	3070 Ti 優化說明
GPU 利用率	> 90%	透過 32 個平行環境與 Large Batch (2048) 餵滿 CUDA Cores。
VRAM 占用	< 6.5 GB	避免 8GB 邊界，預留空間給系統與 TensorBoard 背景運作。
訓練速度 (FPS)	> 12,000 FPS	啟用 AMP 混合精度 + PyTorch 2.0 <code>torch.compile()</code> 加速。
模型收斂性	BipedalWalker Score > 250	使用 Orthogonal 初始化與 GAE 穩定梯度。